

meta-llama / llama-recipes

Public

Scripts for fine-tuning Meta Llama3 with composable FSDP & PEFT methods to cover single/multi-node GPUs. Supports default & custom datasets for applications such as summarization and Q&A. Supporting a number of candid inference solutions such as HF TGI, VLLM for local or cloud deployment. Demo apps to showcase Meta Llama3 for WhatsApp & Messenger.

☆ 10.6k stars

🍴 1.5k forks

🌿 Branches

🏷 Tags

🔄 Activity

☆ Star

🔔 Notifications

<> Code

🔍 Issues 84

🔗 Pull requests 45

🔄 Actions

📁 Projects

🛡 Security

📈 Insights

main

98 Branches

2 Tags

🔍 Go to file

Go to file

Code

...

mreso

Update transformers requirements (#605) ✓

2845306 · yesterday

📁 .github	Update wordlist.txt	yesterday
📁 docs	remove 405B ft doc	2 days ago
📁 recipes	Address feedback not possible before launch in L...	yesterday
📁 src	Address feedback not possible before launch in L...	yesterday
📁 tools/benchmarks	Updates HF model_ids and readmes for 3.1	last week
📄 .gitignore	fix in gitignore	8 months ago
📄 CODE_OF_CONDUCT.md	Initial commit	last year
📄 CONTRIBUTING.md	Update contribution.md	2 months ago
📄 README.md	Update README.md to correct experimental url	yesterday
📄 UPDATES.md	recipes/quickstart folder updated	last month
📄 dev_requirements.txt	Added dev_req.txt files	10 months ago
📄 pyproject.toml	Update transformers requirements (#605)	yesterday
📄 requirements.txt	Update transformers requirements (#605)	yesterday

Llama Recipes: Examples to get started using the Llama models from Meta

The 'llama-recipes' repository is a companion to the [Meta Llama](#) models. We support the latest version, [Llama 3.1](#), in this repository. The goal is to provide a scalable library for fine-tuning Meta Llama models, along with some example scripts and notebooks to quickly get started with using the models in a variety of use-cases, including fine-tuning for domain adaptation and building LLM-based applications with Llama and other tools in the LLM ecosystem. The examples here showcase how to run Llama locally, in the cloud, and on-prem.

Important

Meta Llama 3.1 has a new prompt template and special tokens.

Token	Description
< begin_of_text >	Specifies the start of the prompt.
< eot_id >	This token signifies the end of a turn i.e. the end of the model's interaction either with the user or tool executor.
< eom_id >	End of Message. A message represents a possible stopping point where the model can inform the execution environment that a tool call needs to be made.
< python_tag >	A special tag used in the model's response to signify a tool call.
< finetune_right_pad_id >	Used for padding text sequences in a batch to the same length.

https://github.com/meta-llama/llama-recipes/tree/main

1/5

Token	Description
< start_header_id >{role} < end_header_id >	These tokens enclose the role for a particular message. The possible roles can be: system, user, assistant and ipython.
< end_of_text >	This is equivalent to the EOS token. For multiturn-conversations it's usually unused, this token is expected to be generated only by the base models.

A multiturn-conversation with Meta Llama 3.1 that includes tool-calling follows this structure:

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

{{ system_prompt }}<|eot_id|><|start_header_id|>user<|end_header_id|>

{{ user_message_1 }}<|eot_id|><|start_header_id|>assistant<|end_header_id|>

<|python_tag|>{{ model_tool_call_1 }}<|eom_id|><|start_header_id|>ipython<|end_header_id|>

{{ tool_response }}<|eot_id|><|start_header_id|>assistant<|end_header_id|>

{{model_response_based_on_tool_response}}<|eot_id|>
```

Each message gets trailed by an <|eot_id|> token before a new header is started, signaling a role change.

More details on the new tokenizer and prompt template can be found [here](#).

Note

The llama-recipes repository was recently refactored to promote a better developer experience of using the examples. Some files have been moved to new locations. The `src/` folder has NOT been modified, so the functionality of this repo and package is not impacted.

Make sure you update your local clone by running `git pull origin main`

[README](#) [Code of conduct](#)



Table of Contents

- [Llama Recipes: Examples to get started using the Meta Llama models from Meta](#)
 - [Table of Contents](#)
 - [Getting Started](#)
 - [Prerequisites](#)
 - [PyTorch Nightlies](#)
 - [Installing](#)
 - [Install with pip](#)
 - [Install with optional dependencies](#)
 - [Install from source](#)
 - [Getting the Llama models](#)
 - [Model conversion to Hugging Face](#)
 - [Repository Organization](#)
 - [recipes/](#)
 - [src/](#)
 - [Contributing](#)
 - [License](#)

Getting Started

These instructions will get you a copy of the project up and running on your local machine for development and testing purposes. See deployment for notes on how to deploy the project on a live system.

Prerequisites

PyTorch Nightlies

If you want to use PyTorch nightlies instead of the stable release, go to [this guide](#) to retrieve the right `--extra-index-url` URL parameter for the `pip install` commands on your platform.

Installing

Llama-recipes provides a pip distribution for easy install and usage in other projects. Alternatively, it can be installed from source.

Note

Ensure you use the correct CUDA version (from `nvidia-smi`) when installing the PyTorch wheels. Here we are using 11.8 as `cu118`. H100 GPUs work better with CUDA > 12.0

Install with pip

```
pip install llama-recipes
```

Install with optional dependencies

Llama-recipes offers the installation of optional packages. There are three optional dependency groups. To run the unit tests we can install the required dependencies with:

```
pip install llama-recipes[tests]
```

For the vLLM example we need additional requirements that can be installed with:

```
pip install llama-recipes[vllm]
```

To use the sensitive topics safety checker install with:

```
pip install llama-recipes[auditnl]
```

Optional dependencies can also be combined with `[option1,option2]`.

Install from source

To install from source e.g. for development use these commands. We're using hatchling as our build backend which requires an up-to-date pip as well as setuptools package.

```
git clone git@github.com:meta-llama/llama-recipes.git
cd llama-recipes
pip install -U pip setuptools
pip install -e .
```

For development and contributing to llama-recipes please install all optional dependencies:

```
git clone git@github.com:meta-llama/llama-recipes.git
cd llama-recipes
pip install -U pip setuptools
pip install -e .[tests,auditnl,vllm]
```

Getting the Meta Llama models

You can find Meta Llama models on Hugging Face hub [here](#), where models with `hf` in the name are already converted to Hugging Face checkpoints so no further conversion is needed. The conversion step below is only for original model weights from Meta that are hosted on Hugging Face model hub as well.

Model conversion to Hugging Face

The recipes and notebooks in this folder are using the Meta Llama model definition provided by Hugging Face's transformers library.

Given that the original checkpoint resides under `models/7B` you can install all requirements and convert the checkpoint with:

```
## Install Hugging Face Transformers from source
pip freeze | grep transformers ## verify it is version 4.31.0 or higher

git clone git@github.com:huggingface/transformers.git
cd transformers
pip install protobuf
python src/transformers/models/llama/convert_llama_weights_to_hf.py \
  --input_dir /path/to/downloaded/llama/weights --model_size 7B --output_dir /output/path
```

Repository Organization

Most of the code dealing with Llama usage is organized across 2 main folders: `recipes/` and `src/`.

recipes/

Contains examples are organized in folders by topic:

Subfolder	Description
quickstart	The "Hello World" of using Llama, start here if you are new to using Llama.
use_cases	Scripts showing common applications of Meta Llama3
3p_integrations	Partner owned folder showing common applications of Meta Llama3
responsible_ai	Scripts to use PurpleLlama for safeguarding model outputs
experimental	Meta Llama implementations of experimental LLM techniques

src/

Contains modules which support the example recipes:

Subfolder	Description
configs	Contains the configuration files for PEFT methods, FSDP, Datasets, Weights & Biases experiment tracking.
datasets	Contains individual scripts for each dataset to download and process. Note
inference	Includes modules for inference for the fine-tuned models.
model_checkpointing	Contains FSDP checkpoint handlers.
policies	Contains FSDP scripts to provide different policies, such as mixed precision, transformer wrapping policy and activation checkpointing along with any precision optimizer (used for running FSDP with pure bf16 mode).
utils	Utility files for: - <code>train_utils.py</code> provides training/eval loop and more train utils. - <code>dataset_utils.py</code> to get preprocessed datasets. - <code>config_utils.py</code> to override the configs received from CLI. - <code>fsdp_utils.py</code> provides FSDP wrapping policy for PEFT methods. - <code>memory_utils.py</code> context manager to track different memory stats in train loop.

Contributing

Please read [CONTRIBUTING.md](#) for details on our code of conduct, and the process for submitting pull requests to us.

License

See the License file for Meta Llama 3.1 [here](#) and Acceptable Use Policy [here](#)

Releases 1

 Llama-recipes v.0.0.3 Release Notes

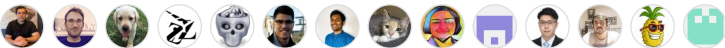
Latest

yesterday

Packages

No packages published

Contributors 74



[+ 60 contributors](#)

Languages

● Jupyter Notebook 87.7% ● Python 12.3%